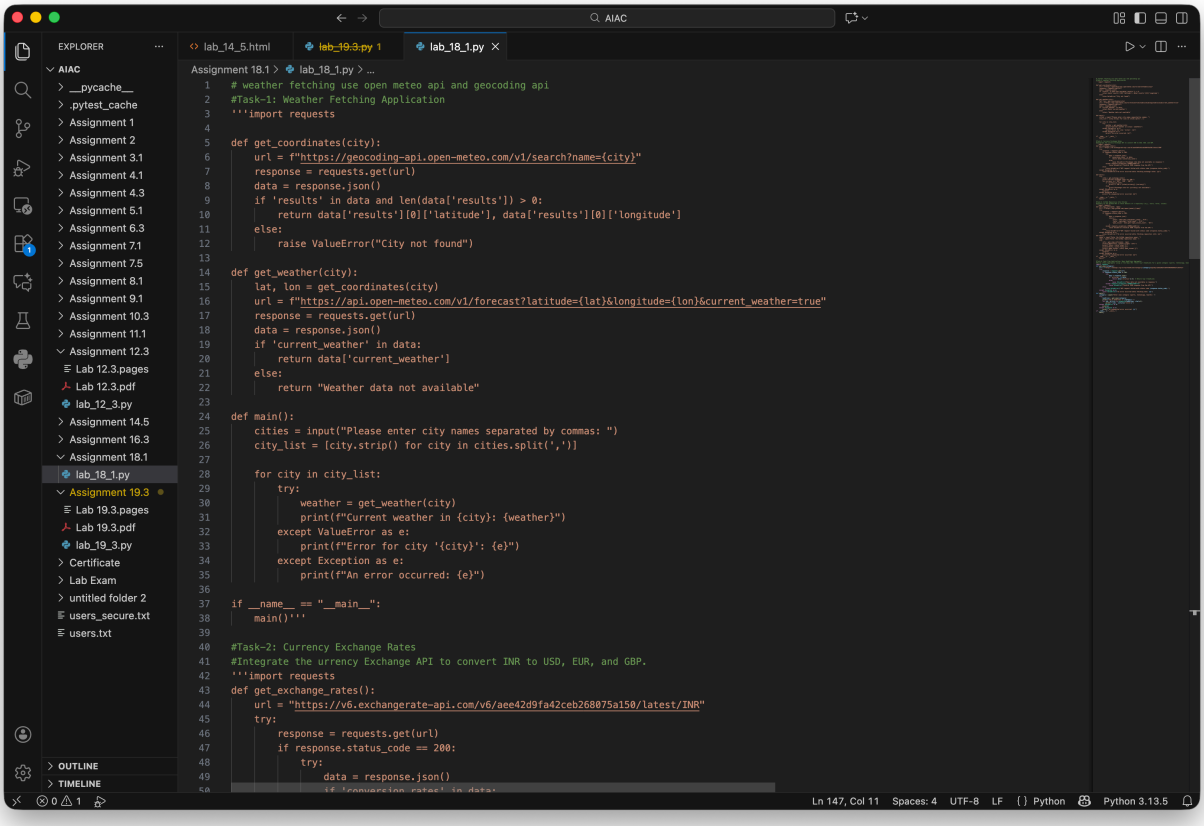
School of Computer Science and Artificial Intelligence

Lab Assignment - 18.1

Course Name : AI Assistant Coding
Name of the Student : Shaik Naved Ahmed
Registration No : 2303A54053
Batch No : 47-B

Task 1 – Weather API Integration
Connect to a Weather API to fetch current weather details for a given city.
Instructions:
- **Prompt AI to generate Python code using the requests library.**
- **Handle errors: invalid city name, API key errors, and network failures.**
- **Display temperature, humidity, and weather description in a readable format.**

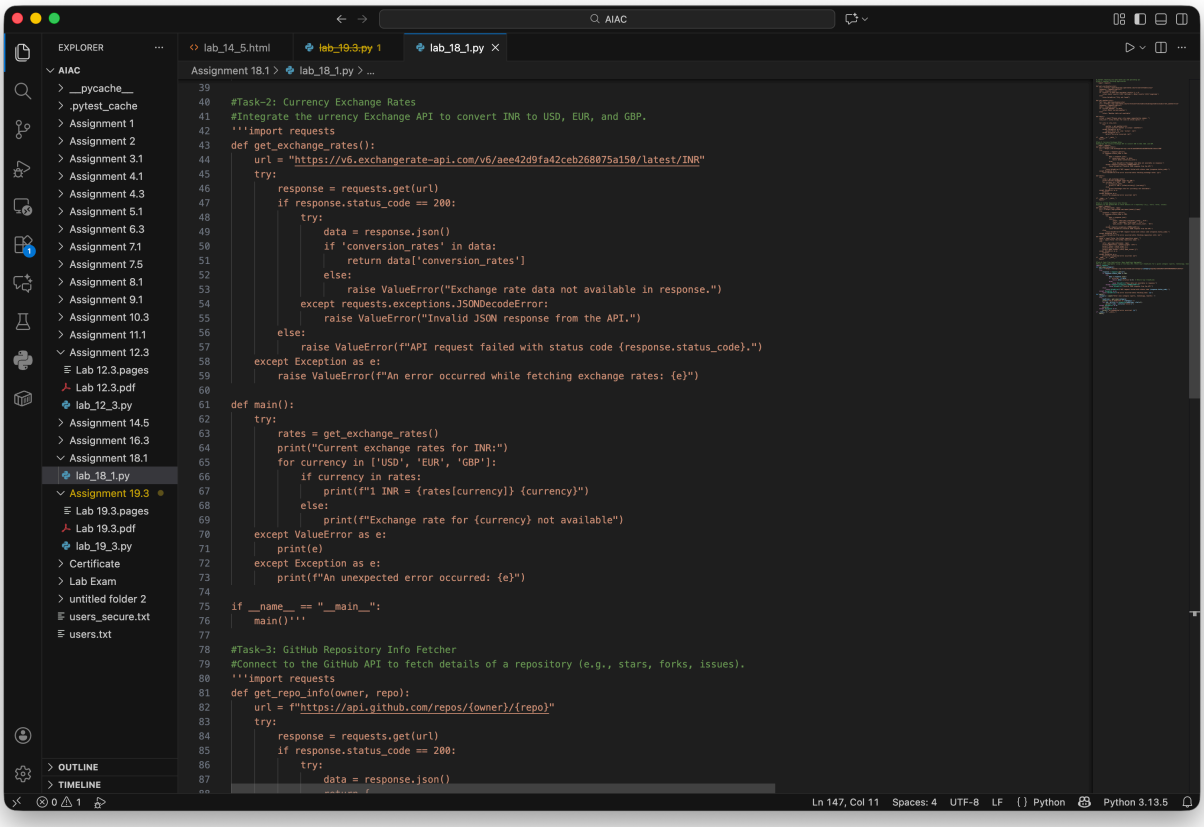
Expected Output:
- **A working Python script that fetches and displays weather data with proper error handling.**



```python
# weather fetching use open meteo api and geocoding api
#Task-1: Weather Fetching Application
'''import requests

def get_coordinates(city):
    url = f"https://geocoding-api.open-meteo.com/v1/search?name={city}"
    response = requests.get(url)
    data = response.json()
    if 'results' in data and len(data['results']) > 0:
        return data['results'][0]['latitude'], data['results'][0]['longitude']
    else:
        raise ValueError("City not found")

def get_weather(city):
    lat, lon = get_coordinates(city)
    url = f"https://api.open-meteo.com/v1/forecast?latitude={lat}&longitude={lon}&current_weather=true"
    response = requests.get(url)
    data = response.json()
    if 'current_weather' in data:
        return data['current_weather']
    else:
        return "Weather data not available"

def main():
    cities = input("Please enter city names separated by commas: ")
    city_list = [city.strip() for city in cities.split(',')]

    for city in city_list:
        try:
            weather = get_weather(city)
            print(f"Current weather in {city}: {weather}")
        except ValueError as e:
            print(f"Error for city '{city}': {e}")
        except Exception as e:
            print(f"An error occurred: {e}")

if __name__ == "__main__":
    main()'''

#Task-2: Currency Exchange Rates
#Integrate the urrency Exchange API to convert INR to USD, EUR, and GBP.
'''import requests
def get_exchange_rates():
    url = "https://v6.exchangerate-api.com/v6/aee42d9fa42ceb268075a150/latest/INR"
    try:
        response = requests.get(url)
        if response.status_code == 200:
            try:
                data = response.json()
```

Task 2 – Currency Exchange Rates
Integrate a Currency Exchange API to convert INR to USD, EUR, and GBP.
Instructions:
- **Use AI to generate API request code.**
- **Validate user input for amount and currency codes.**
- **Implement error handling for invalid responses and API downtime.**
- **Show conversion results clearly in tabular format.**

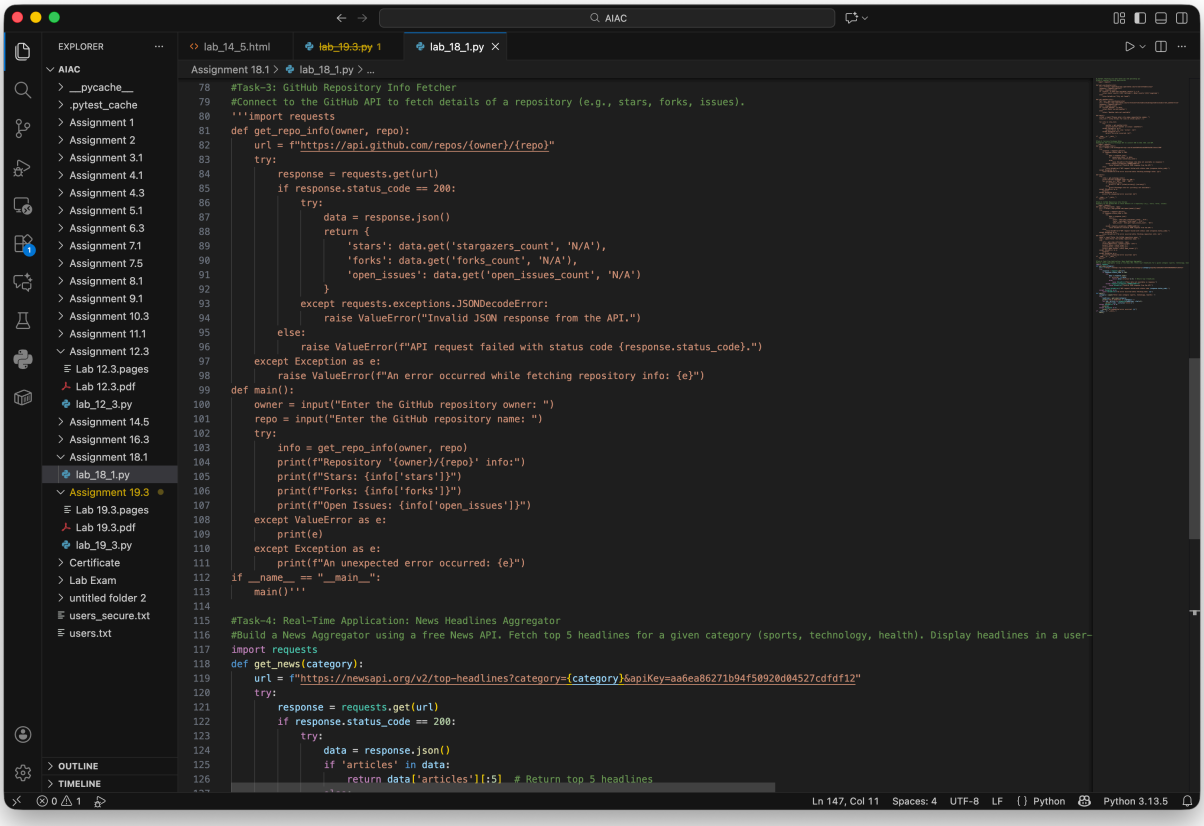
Expected Output:
- **A script that converts INR to multiple currencies with graceful error handling.**



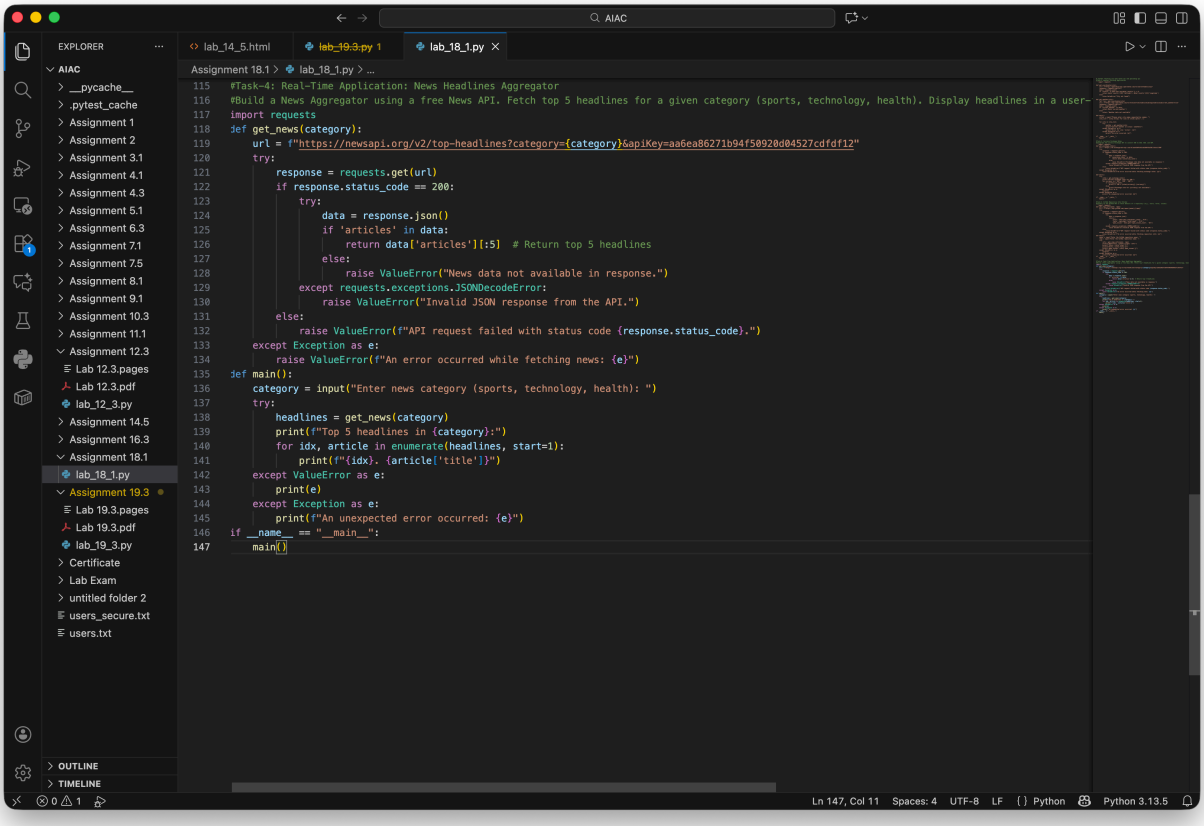
```python
#Task-2: Currency Exchange Rates
#Integrate the urrency Exchange API to convert INR to USD, EUR, and GBP.
'''import requests
def get_exchange_rates():
    url = "https://v6.exchangerate-api.com/v6/aee42d9fa42ceb268075a150/latest/INR"
    try:
        response = requests.get(url)
        if response.status_code == 200:
            try:
                data = response.json()
                if 'conversion_rates' in data:
                    return data['conversion_rates']
                else:
                    raise ValueError("Exchange rate data not available in response.")
            except requests.exceptions.JSONDecodeError:
                raise ValueError("Invalid JSON response from the API.")
        else:
            raise ValueError(f"API request failed with status code {response.status_code}.")
    except Exception as e:
        raise ValueError(f"An error occurred while fetching exchange rates: {e}")

def main():
    try:
        rates = get_exchange_rates()
        print("Current exchange rates for INR:")
        for currency in ['USD', 'EUR', 'GBP']:
            if currency in rates:
                print(f"1 INR = {rates[currency]} {currency}")
            else:
                print(f"Exchange rate for {currency} not available")
    except ValueError as e:
        print(e)
    except Exception as e:
        print(f"An unexpected error occurred: {e}")

if __name__ == "__main__":
    main()'''

#Task-3: GitHub Repository Info Fetcher
#Connect to the GitHub API to fetch details of a repository (e.g., stars, forks, issues).
'''import requests
def get_repo_info(owner, repo):
    url = f"https://api.github.com/repos/{owner}/{repo}"
    try:
        response = requests.get(url)
        if response.status_code == 200:
            try:
                data = response.json()
```

Task 3 – GitHub Repository Info Fetcher
Connect to the GitHub API to fetch details of a repository (e.g., stars, forks, issues).
Instructions:
- **Prompt AI to write a Python function to send GET requests to GitHub API.**
- **Handle rate limit errors, 404 (repo not found), and invalid input.**
- **Display repository name, description, stars, forks, and open issues.**

Expected Output:
- **A Python program that retrieves and displays GitHub repository information with robust error handling.**



```python
#Task-3: GitHub Repository Info Fetcher
#Connect to the GitHub API to fetch details of a repository (e.g., stars, forks, issues).
'''import requests
def get_repo_info(owner, repo):
    url = f"https://api.github.com/repos/{owner}/{repo}"
    try:
        response = requests.get(url)
        if response.status_code == 200:
            try:
                data = response.json()
                return {
                    'stars': data.get('stargazers_count', 'N/A'),
                    'forks': data.get('forks_count', 'N/A'),
                    'open_issues': data.get('open_issues_count', 'N/A')
                }
            except requests.exceptions.JSONDecodeError:
                raise ValueError("Invalid JSON response from the API.")
        else:
            raise ValueError(f"API request failed with status code {response.status_code}.")
    except Exception as e:
        raise ValueError(f"An error occurred while fetching repository info: {e}")
def main():
    owner = input("Enter the GitHub repository owner: ")
    repo = input("Enter the GitHub repository name: ")
    try:
        info = get_repo_info(owner, repo)
        print(f"Repository '{owner}/{repo}' info:")
        print(f"Stars: {info['stars']}")
        print(f"Forks: {info['forks']}")
        print(f"Open Issues: {info['open_issues']}")
    except ValueError as e:
        print(e)
    except Exception as e:
        print(f"An unexpected error occurred: {e}")
if __name__ == "__main__":
    main()'''

#Task-4: Real-Time Application: News Headlines Aggregator
#Build a News Aggregator using a free News API. Fetch top 5 headlines for a given category (sports, technology, health). Display headlines in a user-
import requests
def get_news(category):
    url = f"https://newsapi.org/v2/top-headlines?category={category}&apiKey=aa6ea86271b94f50920d04527cdfdf12"
    try:
        response = requests.get(url)
        if response.status_code == 200:
            try:
                data = response.json()
                if 'articles' in data:
                    return data['articles'][:5]  # Return top 5 headlines
```

Task 4 – Real-Time Application: News Headlines Aggregator
Scenario: Build a News Aggregator using a free News API.
Requirements:
- **Fetch top 5 headlines for a given category (sports, technology, health).**
- **Handle errors like invalid category, missing API key, and request failures.**
- **Display headlines in a user-friendly numbered list format.**
- **Include a retry mechanism if the first request fails.**

Expected Output:
- **A working Python script that retrieves and displays news headlines with proper error handling**